Bringing the agent branch into main.

From agents in chat channels to controlled execution, replayable work and explainable authorization.

01 · Run in the cloud

Sessions, channel policies, events, data and deployment.

02 · Retain the work

Bounded execution, trajectories, artifacts and embedded agents.

03 · Explain authority

Typed queries, SQLite hosts and policy proof trees.

04 · Go native

Zig interpreter, terminal and Git; a separate compatibility track.

89 commits after review and history cleanup

4 proposed review slices with explicit merge gates

This deck separates implemented work, hard problems solved, and decisions still open.

Give every conversation a bounded agent and an explicit policy.

Plugins publish native input types; checked handlers choose sessions and work.

Native input

Plugin-owned value
Published EYG type

→

Checked handler

Select session + work
Bounded history + reset

→

Overlay agent

Bounded turns
Produces EYG programs

→

Channel shell

Gate each effect
Then execute on host

WHAT DEVELOPS

Complete policies are checked when set. Start with `cloud.reject_all` and override gates. CLI and cloud share the decision core.

HARDEST PROBLEM

Keep native message data intact while sharing sessions and work. Preserve conversation history; discard lexical bindings after each execution.

> **Merge review:** trace native input → checked handler → session work → effect gate; check reset and inherited authority.

Expand what agents can do without losing the enforcement point.

Connections, persistent workspace resources and a control surface grow around the same shell.

INPUTS

React to the world

GitHub, feeds, Bluesky, HN and Lobsters; webhooks and Sentry; triggers and scheduled messages.

RESOURCES

Keep useful work

Workspace secrets, session-backed imports, skills and file-change tracking, channel-owned artifacts.

OPERATIONS

Inspect and configure

A policy-gated API and web shell, with OpenTelemetry spans around programs and effects.

Hardest problem: keep active policy and persisted policy consistent. A failed user-policy write must leave the in-memory registry unchanged.

Merge review: inspect state transitions, channel ownership and secret handling. Shared artifact validation now has one implementation in the hub.

Evidence: failed-policy-write regression · trigger events · tracing

Cloud databases pull part of the language roadmap forward.

Move the minimum complete query foundation ahead of deployment; add later host integrations in their own stages.

DEPENDENCY ORDER IN THE REWRITTEN BRANCH

Typed query tables + indexed fixed-point evaluation

↓

SQLite planner + shared effect interfaces

↓

Workspace databases + policy-gated dataclips

The cloud handles SQLite effects directly.
It does not depend on the later local CLI host implementation.

HARDEST PROBLEM / PRESERVE SET SEMANTICS

An empty record is still one fact.

A zero-field relation needs a unique unit column in SQLite. Otherwise repeated derivation can insert duplicates and prevent a fixed point.

```
"$unit" INTEGER UNIQUE DEFAULT 0
```

Seed and deduplicate the same unit value.
Planner and cloud regressions cover this case.

Merge review: check table semantics, SQL seeding and rule scheduling together.

Prove the cloud can stand on its own.

The first review slice ends with tests, an exported BEAM image and a verified private Railway deployment.

EVIDENCE RECORDED DURING BRANCH REVIEW

157
cloud tests after the native-input policy review

65 / 65
jobs in the prior CI run; rerun after publishing

288 / 288
EYG tests on the rewrite, run from the repository root

Healthy
pathfinder / cloud deployment; Docker boot and HTTP checks

Verification also exposed CI-only gaps: missing Rebar3, Hex rate limits, stale guide indexing and binary files in the guide loader.

HARDEST PROBLEM / REPRODUCIBLE INTEGRATION

Local tools and caches hid missing CI setup. Dependencies, manifests, generated runtime assets and browser workflows had to agree at the same revision.

> **Rollout gate:** keep the service private until real authentication fronts it. The example login accepts a name; the first user becomes admin.

Manual GitHub deployment needs a project-scoped `RAILWAY_TOKEN` in environment `pathfinder`. CLI deployment was exercised; the workflow deploy job was skipped on push.

Make execution interruptible without changing what programs mean.

Bounded runs, retained shell cells, compiled execution and tracing must share one semantic model.

Typed cell

Keep lexical binding order
Retain session state

→

Execution slice

Reduction budget
Resume continuation

→

Effect boundary

Handlers and masks
Record + trace outcome

↺

WHAT DEVELOPS

Recorded effects, OTLP export, function profiles, shallow handlers, scoped masks and a retained typed shell exposed through a host API.

HARDEST PROBLEM

Queries and profile returns must survive suspension. Compiled query callbacks must enter the trampoline; standalone modules need the same query builtins.

> **Merge review:** follow one suspended effect through interpreter and compiler. Check callback resumption, handler scope and replay order against the spec fixtures.

Carry the work between tools without carrying hidden authority.

Trajectories, sandbox acquisition and host adapters extend the execution model beyond one chat UI.

RECORD

Replayable sessions

Pinned CLI trajectories, retained browser chats, verified replay and secret redaction.

ISOLATE

Inspect and evaluate

Sealed acquisition, isolated forks, offline evaluations and versioned artifact workspaces.

EMBED

Reuse the boundary

Portable model transports, checked agents, EYG scripts, Pi / OpenCode policies and host examples.

Hardest problem: make archives portable and verifiable while preserving secret redaction, captured query values and artifact ownership.

Merge review: follow acquire → run → export → replay across a real host. Check that native tools use the policy too; a second unrestricted tool path defeats the boundary.

Explain why an operation is allowed.

Local SQLite hosts, executable policy examples and proof trees build on the earlier typed query foundation.

ILLUSTRATIVE EXPLANATION SHAPE

Allow submitter to act on a repository

> Trusted identity mapping
>
> > Submitter belongs to the required channel
>
> Policy rule connects membership to permission

A conceptual reading of the channel-policy example, not a literal proof-tree serialization.

Roles + delegation | SQLite movie demo | Editor highlighting

HARDEST PROBLEM / TRUST THE INPUTS

A derivation explains a result. The host must establish authority.

Typed facts and successful guards are useful evidence. They do not authenticate a caller or prove where facts came from.

> **Merge review:** verify trusted fact construction, tenant / resource scope and the effect-time check. The channel integration is a mocked prototype.

Read: authorization tutorial · proof trees · channel-policy prototype

Treat the Zig runtime as its own merge decision.

Native evaluation, structural terminal UI, Git and an Android host arrive with pinned-oracle evidence.

WHAT DEVELOPS

1. **Native interpreter + inference**
   Parser, effects, continuations and Unicode operations in Zig.
2. **Terminal and Git runtime**
   Differential fixtures, event-by-event UI checks and measured performance reports.
3. **Android + implementation material**
   JNI / Expo host, book, source listings and an oracle-testing talk.

HARDEST PROBLEM / DEFINE PARITY PRECISELY

The oracle is pinned to `be502aa` on `next-tui`. This runtime does **not** support newer root-spec Datalog queries.

Evidence limit: this branch review reran native unit tests and a release smoke test. It did not rerun the full differential campaigns or assemble an Android APK.

Merge decision: accept an explicitly experimental package, or land it separately after refreshed parity and device evidence.

A concrete path into main.

Use these as sequential PR boundaries, or as four approval gates within one PR.

REVIEW SLICE	ACCEPTANCE FOCUS	END CHECKPOINT
A · Cloud + query core	Session / policy transitions, SQLite semantics, persistent image and private deployment.	12a4047 `cloud deployment`
B · Execution + tooling	Continuation parity, replay / redaction, actual host policy enforcement and browser workflows.	2e9560f `integration / query design`
C · Policy applications	Local SQLite behavior, trusted facts and proof claims; keep mocked examples clearly labeled.	653a2d8 `channel policies + proofs`
D · Native track	Decide experimental scope or a separate PR; document oracle gaps and Android evidence.	ea23c6e `native work + roadmap`

Before merge: compare with current main, resolve any drift, and run required checks on the actual candidate. Assign reviewers for runtime semantics, cloud state and the native track.

Before public rollout: provide authentication, exercise the GitHub deployment path with its scoped token, and verify persistence / recovery for the deployed workspace.